Foundations of Perceptron

[Perceptron]

$$y = \operatorname{sign}(w \cdot x + b)$$

$$w \leftarrow w + \eta(y - \hat{y})x$$

1.0 Content Block

While the perceptron algorithm is guaranteed to converge on some solution in the case of a linearly separable training set, it may still pick any solution and problems may admit many solutions of varying quality. The perceptron of optimal stability, nowadays better known as the linear support-vector machine, was designed to solve this problem (Krauth and Mezard, 1987).

2.0 Content Block

Below is an example of a learning algorithm for a single-layer perceptron with a single output unit. For a single-layer perceptron with multiple output units, since the weights of one output unit are completely separate from all the others', the same algorithm can be run for each output unit. For multilayer perceptrons, where a hidden layer exists, more sophisticated algorithms such as backpropagation must be used. If the activation function or the underlying process being modeled by the perceptron is nonlinear, alternative learning algorithms such as the delta rule can be used as long as the activation function is differentiable. Nonetheless, the learning algorithm described in the steps below will often work, even for multilayer perceptrons with nonlinear activation functions. When multiple perceptrons are combined in an artificial neural network, each output neuron operates independently of all the others; thus, learning each output can be considered in isolation.

3.0 Content Block

Information theory From an information theory point of view, a single perceptron with K inputs has a capacity of 2K bits of information. This result is due to Thomas Cover. Specifically let T (N , K) {\displaystyle T(N,K)} be the number of ways to linearly separate N points in K dimensions, then T (N , K) = { 2 N K ≥ N 2 ∑ k = 0 K − 1 (N − 1 k) K < N {\displaystyle T(N,K)=\left\{{\begin{array}{cc}2^{N}&K\geq N\\2\sum _{k=0}^{K-1}\left({\begin{array}{c}N-1\\k\end{array}}\right)&K<N\end{array}}\right.} When K is large, T (N , K) / 2 N {\displaystyle T(N,K)/2^{N}} is very close to one when N ≤ 2 K {\displaystyle N\leq 2K} , but very close to zero when N > 2 K {\displaystyle N>2K} . In words, one perceptron unit can almost certainly memorize a random assignment of binary labels on N points when N ≤ 2 K {\displaystyle N\leq 2K} , but almost certainly not when N > 2 K {\displaystyle N>2K} .

4.0 Content Block

A single perceptron can learn to classify any half-space. It cannot solve any linearly nonseparable vectors, such as the Boolean exclusive-or problem (the famous "XOR problem"). A perceptron network with one hidden layer can learn to classify any compact subset arbitrarily closely. Similarly, it can also approximate any compactly-supported continuous function arbitrarily closely. This is essentially a special case of the theorems by George Cybenko and Kurt Hornik.

5.0 Content Block

Further reading Aizerman, M. A. and Braverman, E. M. and Lev I. Rozonoer. Theoretical foundations of the potential function method in pattern recognition learning. Automation and Remote Control, 25:821–837, 1964. Rosenblatt, Frank (1958), The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain, Cornell Aeronautical Laboratory, Psychological Review, v65, No. 6, pp. 386–408. doi:10.1037/h0042519. Rosenblatt, Frank (1962), Principles of Neurodynamics. Washington, DC: Spartan Books. Minsky, M. L. and Papert, S. A. 1969. Perceptrons. Cambridge, MA: MIT Press. Gallant, S. I. (1990). Perceptron-based learning algorithms. IEEE Transactions on Neural Networks, vol. 1, no. 2, pp. 179–191. Olazaran Rodriguez, Jose Miguel. A historical sociology of neural network research. PhD Dissertation. University of Edinburgh, 1991. Mohri, Mehryar and Rostamizadeh, Afshin (2013). Perceptron Mistake Bounds arXiv:1305.0208, 2013. Novikoff, A. B. (1962). On convergence proofs on perceptrons. Symposium on the Mathematical Theory of Automata, 12, 615–622. Polytechnic Institute of Brooklyn. Widrow, B., Lehr, M.A., "30 years of Adaptive Neural Networks: Perceptron, Madaline, and Backpropagation," Proc. IEEE, vol 78, no 9, pp. 1415–1442, (1990). Collins, M. 2002. Discriminative training methods for hidden Markov models: Theory and experiments with the perceptron algorithm in Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP '02). Yin, Hongfeng (1996), Perceptron-Based Algorithms and Analysis, Spectrum Library, Concordia University, Canada

6.0 Content Block

Boolean function When operating on only binary inputs, a perceptron is called a linearly separable Boolean function, or threshold Boolean function. The sequence of numbers of threshold Boolean functions on n inputs is OEIS A000609. The value is only known exactly up to n = 9 {\displaystyle n=9} case, but the order of magnitude is known quite exactly: it has upper bound 2 n 2 − n log 2 ■ n + O (n) {\displaystyle 2^{n^{2}-n\log _{2}n+O(n)}} and lower bound 2 n 2 − n log 2 ■ n − O (n) {\displaystyle 2^{n^{2}-n\log _{2}n-O(n)}} . Any Boolean linear threshold function can be implemented with only integer weights. Furthermore, the number of bits necessary and sufficient for representing a single integer weight parameter is Θ (n ln ■ n) {\displaystyle \Theta (n\ln n)} .

7.0 Content Block

The binary value of f (x) {\displaystyle f(\mathbf {x})} (0 or 1) is used to perform binary classification on x {\displaystyle \mathbf {x} } as either a positive or a negative instance. Spatially, the bias shifts the position (though not the orientation) of the planar decision boundary. In the context of neural networks, a perceptron is an artificial neuron using the Heaviside step function as the activation function. The perceptron algorithm is also termed the single-layer perceptron, to distinguish it from a multilayer perceptron, which is a misnomer for a more complicated neural network. As a linear classifier, the single-layer perceptron is the simplest feedforward neural network.

8.0 Content Block

Learning again iterates over the examples, predicting an output for each, leaving the weights unchanged when the predicted output matches the target, and changing them when it does not. The update becomes:

9.0 Content Block

d j ∈ { 0 , 1 } {\displaystyle d_{j}\in \{0,1\}} is the desired output value of the perceptron for that input. We show the values of the features as follows:

10.0 Content Block

where h {\displaystyle h} is the Heaviside step-function (where an input of > 0 {\textstyle >0} outputs 1; otherwise 0 is the output), w {\displaystyle \mathbf {w} } is a vector of real-valued weights, w ⋅ x {\displaystyle \mathbf {w} \cdot \mathbf {x} } is the dot product ∑ i = 1 m w i x i {\textstyle \sum

_{i=1}^{m}w_{i}x_{i}} , where m is the number of inputs to the perceptron, and b is the bias. The bias shifts the decision boundary away from the origin and does not depend on any input value. Equivalently, since w · x + b = (w , b) · (x , 1) {\displaystyle \mathbf {w} \cdot \mathbf {x} +b=(\mathbf {w} ,b)\cdot (\mathbf {x} ,1)} , we can add the bias term b {\displaystyle b} as another weight w m + 1 {\displaystyle \mathbf {w} _{m+1}} and add a coordinate 1 {\displaystyle 1} to each input x {\displaystyle \mathbf {x} } , and then write it as a linear classifier that passes the origin: f (x) = h (w · x) {\displaystyle f(\mathbf {x})=h(\mathbf {w} \cdot \mathbf {x})}